

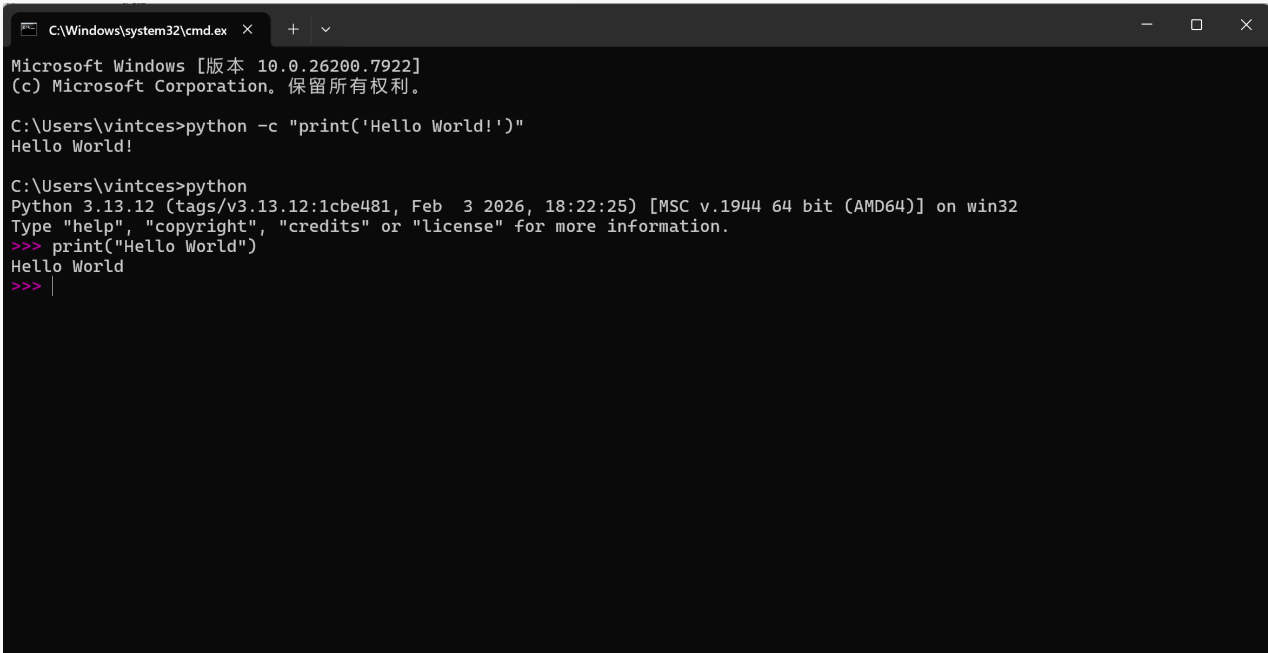
多媒体第一次实验

1.完成本次实验任务的思路

本实验主要分为环境搭建、交互式编程练习以及结构化脚本编写三个阶段：

- **环境构建**：安装 Python 3.7.6 或更高版本。
- **多模式交互验证**：通过 Windows 命令行、Python Shell 以及 `.py` 脚本文件三种方式运行 "Hello World"，熟悉解释器在不同入口下的执行逻辑。
- **模块化编程热身**：
 - 针对给定的 `list` 数据，在 `main.py` 中设计五个独立函数。
 - **算术处理**：实现平均值计算（累加后除以长度）和最大值查找（迭代比较）。
 - **算法实现**：编写冒泡排序，通过双层循环实现元素的从小到大排列。
 - **容器操作**：利用 Python 内置的索引机制，实现指定位置的元素插入与删除。

2.运行程序截图和简要说明

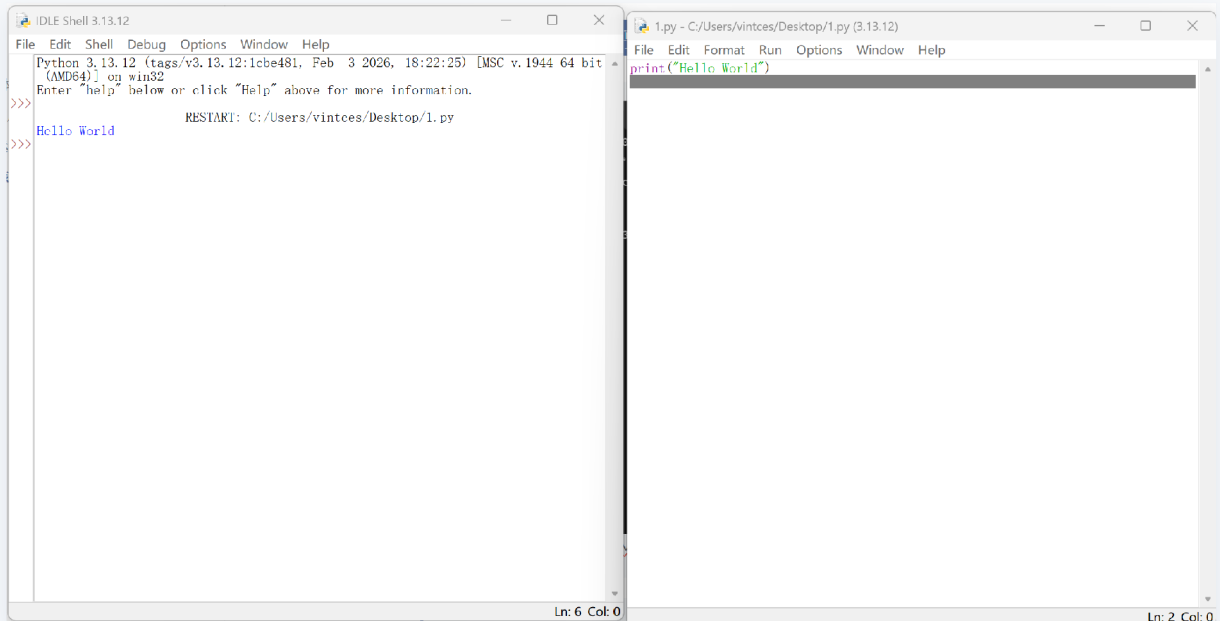


```
C:\Windows\system32\cmd.exe X + v
Microsoft Windows [版本 10.0.26200.7922]
(c) Microsoft Corporation. 保留所有权利。

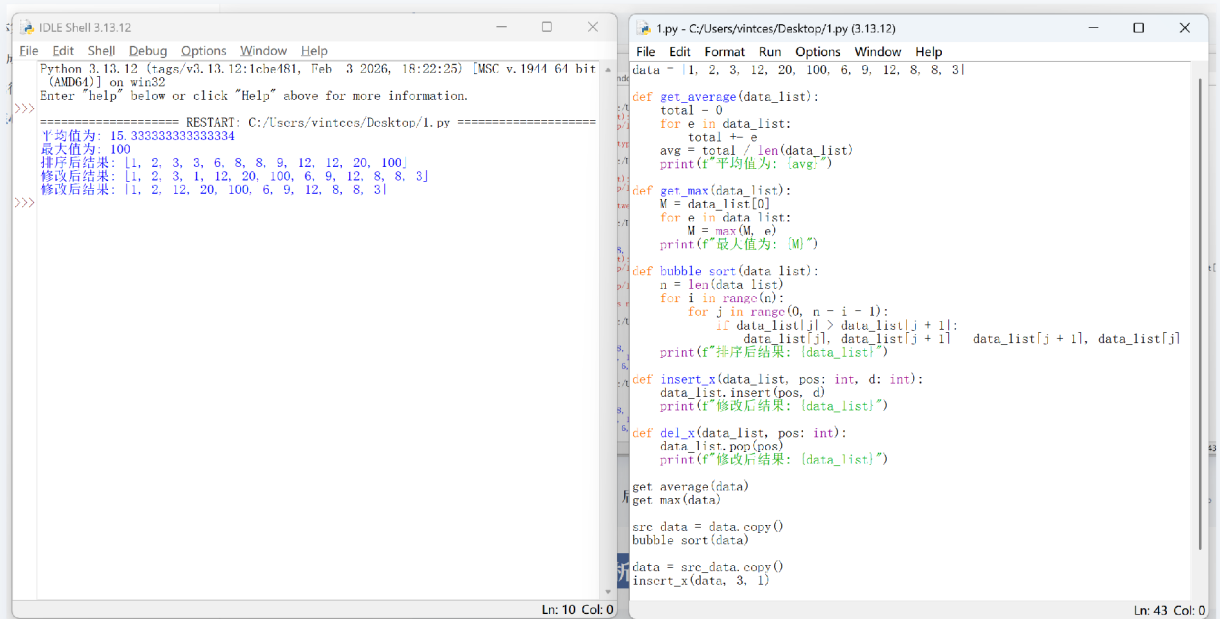
C:\Users\vintces>python -c "print('Hello World!')"
Hello World!

C:\Users\vintces>python
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb 3 2026, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World")
Hello World
>>> |
```

说明：展示在 Windows CMD 和 Python Shell 中成功输出 "Hello World" 的界面。



说明：展示用 Python IDLE 新建一个 `hello.py` 文件并运行 Hello World



说明：展示运行 `main.py` 后，控制台按顺序输出的平均值、最大值、排序后的列表以及插入、删除元素后的列表。

3.核心代码展示和分析

```
data = [1, 2, 3, 12, 20, 100, 6, 9, 12, 8, 8, 3]
```

```

def get_average(data_list):
    total = 0
    for e in data_list:
        total += e
    avg = total / len(data_list)
    print(f"平均值为: {avg}")

def get_max(data_list):
    M = data_list[0]
    for e in data_list:
        M = max(M, e)
    print(f"最大值为: {M}")

def bubble_sort(data_list):
    n = len(data_list)
    for i in range(n):
        for j in range(0, n - i - 1):
            if data_list[j] > data_list[j + 1]:
                data_list[j], data_list[j + 1] = data_list[j + 1], data_list[j]
    print(f"排序后结果: {data_list}")

def insert_x(data_list, pos: int, d: int):
    data_list.insert(pos, d)
    print(f"修改后结果: {data_list}")

def del_x(data_list, pos: int):
    data_list.pop(pos)
    print(f"修改后结果: {data_list}")

```

通过遍历列表累加所有元素值并除以列表长度，计算并输出数据的算术平均值。

```
get_average(data)
```

利用迭代比较的方法遍历列表，实时更新并记录当前发现的最大数值。

```
get_max(data)
```

通过 bubble_sort 函数实现了冒泡排序算法，使用双层循环对比相邻元素并交换，确保列表按升序排列。

使用 data.copy() 保证了每次操作（插入、删除）均在原始数据基础上进行，互不干扰。

```
src_data = data.copy()
```

```
bubble_sort(data)
```

调用列表内置的 insert 方法，在指定的索引位置处插入特定的数据元素。

```
data = src_data.copy()
```

```
insert_x(data, 3, 1)
```

利用列表的 pop 方法，根据提供的索引值移除对应位置的元素并更新列表。

```
data = src_data.copy()
```

```
del_x(data, 2)
```

